

中山大學
SUN YAT-SEN UNIVERSITY

并行程序设计与算法 | 实验十一

CUDA 卷积神经网络算子实现与性能分析

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

课 程 教 师: 吴迪、刘学正

起 始 日 期: 2026 年 6 月 10 日

结 束 日 期: 2026 年 6 月 16 日

实 验 地 点: B202

目录

1 实验简述 **2**
1.1 实验内容 . . . 2
1.2 实验项目结构 . . . 2
1.3 环境与使用说明 . . . 2

2 分析设计 **3**
2.1 直接卷积的数学映射与 GPU 线程组织 . . . 3
2.2 im2col 的数学映射原理 . . . 4
2.3 im2col + GEMM 协同工作的算法设计 . . . 4
2.4 访存模式与性能预期分析 . . . 4

3 核心代码 **5**
3.1 直接卷积 Kernel —2D Grid + 2D Block . . . 5
3.2 im2col Kernel —1D Grid + 1D Block . . . 6
3.3 Shared Memory Tiled GEMM Kernel . . . 7
3.4 cuDNN 卷积调用 . . . 8

4 结果分析 **9**
4.1 实验过程展示 . . . 9
4.2 性能数据汇总 . . . 11
4.3 可视化图表分析 . . . 12
4.4 定量与定性分析 . . . 13
4.5 硬件架构层面深度剖析 . . . 14
4.5.1 为什么 Direct Conv 在极小 Kernel 上近乎最优? . . . 14
4.5.2 im2col+GEMM 的适用边界 . . . 14
4.5.3 cuDNN 算法选择的观察 . . . 15
4.6 Checksum 校验与数值正确性 . . . 15

5 实验总结 **15**
5.1 实验收获 . . . 15
5.2 踩坑与反思 . . . 16

1 实验简述

1.1 实验内容

卷积神经网络（CNN）计算核心——卷积运算——具有高计算密度与丰富的数据复用特性，天然适合 GPU 并行加速。本实验要求使用 CUDA 编程模型，实现并对比三种卷积计算方法的性能：

1. **CUDA 直接卷积 (Direct Convolution)**：采用 2D 滑动窗口法，在 GPU 上通过 2D Grid + 2D Block 线程组织实现 Naive 直接卷积。每个线程负责输出特征图一个空间位置的所有通道计算。

2. **im2col + GEMM 卷积**：利用经典的 im2col (Image to Column) 方法将 3D 输入张量转换为 2D 矩阵，将卷积核整形为矩阵后，调用共享内存分块的 GEMM Kernel 完成卷积。这是早期深度学习框架（如 Caffe）的核心策略。

3. **cuDNN 卷积**：调用 NVIDIA cuDNN 库的标准卷积 API，通过 `cudnnConvolutionForward()` 使用其内部自适应的算法选择器完成相同配置的卷积操作，作为工业级基线。

实验对比变量：

- 输入尺寸 $H = W \in \{32, 64, 128, 256, 512, 1024, 2048\}$（方形输入）。
- 卷积步长（Stride）$s \in \{1, 2, 3\}$。
- 方法类型：Direct / im2col+GEMM / cuDNN。

固定参数：输入通道 $C = 3$，输出通道 $C_{\text{out}} = 3$，卷积核尺寸 $K = 3 \times 3$，填充 $p = 1$，无偏置 $(\text{bias} = 0)$。

1.2 实验项目结构

```
lab11/
    src/
        convolution.cu            -- CUDA 核心源码 (3 种卷积方法)
        Makefile                   -- NVCC 编译自动化
        benchmark_results.csv      -- 基准测试输出数据 (21 组)
        plot_benchmark.py          -- Python 可视化脚本
    img/
        SYSULogo.png
        fig1_execution_time.png    -- 执行时间对比图
        fig2_speedup.png           -- 加速比对比图
        fig3_stride_comparison.png -- Stride 方法对比图
    lab11.pdf                      -- 本实验报告
```

1.3 环境与使用说明

实验编译与运行环境：

- 操作系统：WSL2 Ubuntu 22.04（x86_64，内核 6.6）；

- GPU：NVIDIA GeForce RTX 4060 Laptop GPU（8 GB VRAM，计算能力 8.9）；
- CUDA 驱动：555.97（CUDA 12.5）；
- NVCC：NVIDIA CUDA 12.0.140；
- cuDNN：8.9.2；
- GCC：13.3.0；
- Python：3.12.3 + matplotlib 3.6.3 + numpy 1.26.4；
- 编译链接选项：`nvcc -O3 -arch=sm_89 -lcudnn -o convolution convolution.cu`。

执行方式：

- 编译：进入 `src/` 执行 `make all`；
- 单点运行：`./convolution <input_size> <stride> [csv_file]`；
- 全量测试：手动执行 7 种尺寸 × 3 种 Stride 共 21 组，或使用 `make run-csv`（预设只覆盖前 5 种尺寸，大尺寸需单独追加）；
- 图表生成：`python3 src/plot_benchmark.py`（解析 `benchmark_results.csv` 数据到 `img/` 目录）。

2 分析设计

2.1 直接卷积的数学映射与 GPU 线程组织

对于输入张量 $\mathbf{I} \in \mathbb{R}^{C\times H\times W}$、卷积核 $\mathbf{K} \in \mathbb{R}^{C_{\text{out}}\times C\times K\times K}$，输出张量 $\mathbf{O} \in \mathbb{R}^{C_{\text{out}}\times H_{\text{out}}\times W_{\text{out}}}$ 中每个元素的数学定义为：

$$\mathbf{O}[oc, oy, ox] = \sum_{ic=0}^{C-1}\sum_{ky=0}^{K-1}\sum_{kx=0}^{K-1} \mathbf{I}[ic,\ oy\cdot s + ky - p,\ ox\cdot s + kx - p] \times \mathbf{K}[oc, ic, ky, kx] \tag{1}$$

其中 p 为 padding，s 为 stride。输出空间尺寸为：

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - K}{s} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W + 2p - K}{s} \right\rfloor + 1 \tag{2}$$

在 GPU 上使用 2D Grid + 2D Block 映射：

$$ox = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x} \tag{3}$$

$$oy = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y} \tag{4}$$

每个线程遍历所有输出通道 oc 和所有输入通道 ic，计算其负责空间位置的内积。Block 尺寸固定为 16×16，Grid 尺寸为 $\lceil W_{\text{out}}/16 \rceil \times \lceil H_{\text{out}}/16 \rceil$。

2.2 im2col 的数学映射原理

im2col 的核心思想是将 3D 卷积操作降维为 2D 矩阵乘法。对于每个输出空间位置 (oy, ox)，感受野覆盖输入中 $C \times K \times K$ 个元素。定义：

- **im2col 矩阵** $\mathbf{I}_{\text{col}} \in \mathbb{R}^{(C \cdot K^2) \times (H_{\text{out}} \cdot W_{\text{out}})}$：每一列对应一个输出位置的展开感受野。
- **卷积核矩阵** $\mathbf{K}_{\text{mat}} \in \mathbb{R}^{C_{\text{out}} \times (C \cdot K^2)}$：每个输出通道的卷积核直接展平为一行。

卷积计算等价为矩阵乘法：

$$\mathbf{O}_{\text{mat}} = \mathbf{K}_{\text{mat}} \times \mathbf{I}_{\text{col}}, \quad \mathbf{O}_{\text{mat}} \in \mathbb{R}^{C_{\text{out}} \times (H_{\text{out}} \cdot W_{\text{out}})} \tag{5}$$

其中索引映射关系为：

$$\mathbf{I}_{\text{col}}[ic \cdot K^2 + ky \cdot K + kx,\ oy \cdot W_{\text{out}} + ox] = \mathbf{I}[ic,\ oy \cdot s + ky - p,\ ox \cdot s + kx - p] \tag{6}$$

当感受野坐标超出输入边界时，im2col 矩阵对应位置填充 0，与直接卷积中跳过越界坐标具有数学等价性（$0 \times x = 0$）。

2.3 im2col + GEMM 协同工作的算法设计

整体流程分为两个 Phase：

Phase 1 —im2col 变换。使用 1D Grid + 1D Block 的 CUDA Kernel 将输入张量转换为 im2col 矩阵。每个线程处理一个矩阵元素，总线程数 $= C \cdot K^2 \cdot H_{\text{out}} \cdot W_{\text{out}}$。对于 $H = W = 2048, s = 1$ 的情况，im2col 矩阵包含 $3 \times 9 \times 2048 \times 2048 \approx 1.13 \times 10^8$ 个元素，内存占用约 452 MB（float32），相比原始输入 ($3 \times 2048^2 \approx 50$ MB) 膨胀约 9 倍。

Phase 2 —Shared Memory Tiled GEMM。采用与实验十相同的共享内存分块矩阵乘法 Kernel。共享内存 Tile 尺寸 $T \times T$（$T = 16$），每个 Block 内的线程协同加载 $\mathbf{K}_{\text{mat}}$ 和 $\mathbf{I}_{\text{col}}$ 的分块到 `__shared__` 数组，经 `__syncthreads()` 后在 Shared Memory 中完成局部点积累加。零填充边界保护确保非整除维度的正确性。

2.4 访存模式与性能预期分析

三种方法的计算量完全相同（均为 $C_{\text{out}} \cdot C \cdot K^2 \cdot H_{\text{out}} \cdot W_{\text{out}}$ 次 MAC），但访存模式差异显著：

1. **Direct Conv**：每次卷积涉及 $C \cdot K^2 = 27$ 次输入读取和 $C \cdot K^2 = 27$ 次 Kernel 读取。输入在相邻输出位置之间存在大量重叠（strided reuse），但 Naive 实现的每个线程独立从全局内存读取，数据复用仅在 L2 Cache 层面发生。

2. **im2col + GEMM**：通过 im2col 将数据复用显式化——每个输入元素被复制到 im2col 矩阵的多个位置（最多 $K^2 = 9$ 次复制）。在 GEMM 阶段，Shared Memory Tiling 实现 Block 级数据复用，全局内存读取量从 $2MNK$ 降至 $K(M+N)$。但 im2col 本身的内存膨胀（9×）和双 Kernel 启动开销是此方法的固有缺陷。

3. **cuDNN**：使用自适应算法选择器，可能选择 IMPLICIT_GEMM、Winograd、或 FFT 卷积。对于 3×3 小 Kernel，Winograd $F(2,3)$ 可将乘法次数降至原始的 $16/36 \approx 44\%$。

性能预期：对于 $K = 3$ 这种极小型 Kernel，直接卷积的计算密度较低（每输出元素仅 27 次 MAC），GPU 的线程级并行度（$H_{\text{out}} \cdot W_{\text{out}}$ 个线程）足以充分利用 SM。im2col+GEMM 的额外内存开销和 Kernel 启动延迟可能抵消 GEMM 阶段的数据复用收益，预计在小至中规模落后于 Direct Conv；cuDNN 在大规模下可能通过 Winograd 或隐式 GEMM 胜出。

3 核心代码

3.1 直接卷积 Kernel —2D Grid + 2D Block

```
__global__ void direct_conv_kernel(
    const float * __restrict__ input,
    const float * __restrict__ kernel,
    float * __restrict__ output,
    int C, int H, int W,
    int C_out, int K,
    int H_out, int W_out,
    int pad, int stride)
{
    int ox = blockIdx.x * blockDim.x + threadIdx.x;
    int oy = blockIdx.y * blockDim.y + threadIdx.y;

    if (ox >= W_out || oy >= H_out) return;  // (1) 边界保护

    int input_plane = H * W;
    int kernel_plane = C * K * K;
    int output_plane = H_out * W_out;

    for (int oc = 0; oc < C_out; oc++) {
        float sum = 0.0f;
        const float *k_ptr = kernel + oc * kernel_plane;  // (2) 通道指针偏移

        for (int ic = 0; ic < C; ic++) {
            int iy_base = oy * stride - pad;  // (3) 感受野基址计算
            int ix_base = ox * stride - pad;

            for (int ky = 0; ky < K; ky++) {
                int iy = iy_base + ky;
                if (iy < 0 || iy >= H) continue;  // (4) 纵向边界检查
                for (int kx = 0; kx < K; kx++) {
                    int ix = ix_base + kx;
                    if (ix < 0 || ix >= W) continue;  // (5) 横向边界检查
                    sum += input[ic * input_plane + iy * W + ix]
                         * k_ptr[ic * K * K + ky * K + kx];
                }
            }
        }
        output[oc * output_plane + oy * W_out + ox] = sum;
    }
}
```

Listing 1: convolution.cu：直接卷积 CUDA Kernel

逐段技术分析：

1. **线程映射与边界保护（行 12-13）**：通过 `blockIdx` 和 `threadIdx` 直接计算输出坐标 (ox, oy)，2D 映射无需整数除法/取模。边界检查 `if (ox >= W_out || oy >= H_out) return` 处理非整除 Grid 情况，使得所有 Block 尺寸选择均能正确工作。

2. **通道指针预计算（行 21）**：`k_ptr` 指向当前输出通道 oc 的 Kernel 子块起始地址，将通道维度的乘法移出内层循环。Kernel 在内存中以 (oc, ic, ky, kx) 布局存储，预计算避免了每轮内积的地址乘法。

3. **感受野基址（行 24-25）**：`iy_base = oy * stride - pad` 和 `ix_base = ox * stride - pad` 计算感受野在输入中的左上角坐标。这是 Stride 和 Padding 参与计算的唯一位置，体现了卷积几何参数与计算逻辑的解耦。

4. **双边边界检查（行 29-30, 34-35）**：每次访存前均执行 `ix >= 0 && ix < W` 和 `iy >= 0 && iy < H`。对于 Padding 填充区域（越界坐标），`continue` 跳过读取，等价于零填充但与直接卷积的语义严格一致。此设计避免了 im2col 的内存膨胀，但引入了一定量的 Warp Divergence——越界线程执行 `continue` 路径而界内线程继续计算。

5. **访存模式分析**：`input[ic * input_plane + iy * W + ix]` 中，同一 Warp 内线程的 ox 递增、oy 相同，因此：

   - **I** 的访问：ox 递增 $\Rightarrow$ ix 递增 $\Rightarrow$ Warp 内地址连续 $\Rightarrow$ **合并读 (Coalesced Read)**。
   - **K** 的访问：与 ox, oy 无关，同一 Warp 内所有线程访问相同的 Kernel 元素 $\Rightarrow$ **广播 (Broadcast)**。
   - **O** 的写入：ox 递增 $\Rightarrow$ 地址连续 $\Rightarrow$ **合并写 (Coalesced Write)**。

   全程无非合并访问，这是直接卷积在 GPU 上的天然优势。

3.2 im2col Kernel —1D Grid + 1D Block

```
__global__ void im2col_kernel(
    const float * __restrict__ input,
    float * __restrict__ col,
    int C, int H, int W,
    int K, int H_out, int W_out,
    int pad, int stride)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int total = C * K * K * H_out * W_out;
    if (idx >= total) return;                    // (1) 总量边界

    int col_stride = H_out * W_out;              // (2) 列跨步
    int patch_idx  = idx / col_stride;           // Patch 内偏移
    int sp_idx     = idx % col_stride;           // 空间位置

    int ox = sp_idx % W_out;
    int oy = sp_idx / W_out;

    int ic = patch_idx / (K * K);                // (3) 通道/Kernel 反向映射
    int kk = patch_idx % (K * K);
    int ky = kk / K;
    int kx = kk % K;

    int ix = ox * stride + kx - pad;             // (4) 输入坐标计算
    int iy = oy * stride + ky - pad;

    if (ix >= 0 && ix < W && iy >= 0 && iy < H)
        col[idx] = input[ic * H * W + iy * W + ix];
    else
        col[idx] = 0.0f;                         // (5) 零填充
}
```

Listing 2: convolution.cu：im2col 变换 Kernel

设计要点：

(1) 使用 1D Grid 和 1D Block（Block Size = 256），总线程数覆盖 im2col 矩阵的所有元素。每个线程计算一个矩阵元素，映射简单且无同步需求。

(2) `col_stride = H_out * W_out` 表示 im2col 矩阵的列数。线性索引 `idx = patch_idx * col_stride + sp_idx` 中，`patch_idx` 在 $[0, C \cdot K^2)$ 范围内变化，`sp_idx` 在 $[0, H_out \cdot W_out)$ 范围内变化。

(3) 反向映射通过整数除法将线性 patch 索引分解为 (ic, ky, kx) 三元组。由于 $K = 3$ 且 $C = 3$，K^2 仅为 9，整数除法开销较小。

(4) 输入坐标计算与直接卷积完全一致：$ix = ox \cdot s + kx - p$。这保证了 im2col+GEMM 路径与直接卷积的数学等价性。

(5) 越界坐标以 $0.0f$ 填充。在后续 GEMM 的 $sum \mathrel{+}= A[row, k] \times B[k, col]$ 中，$0.0f \times x = 0.0f$ 保证零填充对累加结果无污染。且所有线程执行相同指令路径，无 Warp Divergence。

3.3 Shared Memory Tiled GEMM Kernel

```cuda
#define TILE_SIZE 16

__global__ void gemm_tiled_kernel(
    const float * __restrict__ A,    // [M, K_gemm]
    const float * __restrict__ B,    // [K_gemm, N]
    float * __restrict__ C,          // [M, N]
    int M, int N, int K_gemm)
{
    __shared__ float As[TILE_SIZE][TILE_SIZE];
    __shared__ float Bs[TILE_SIZE][TILE_SIZE];

    int row = blockIdx.y * TILE_SIZE + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;
    float sum = 0.0f;

    int num_tiles = (K_gemm + TILE_SIZE - 1) / TILE_SIZE;
    for (int t = 0; t < num_tiles; t++) {
        // 加载 A 分块
        int a_col = t * TILE_SIZE + threadIdx.x;
        if (row < M && a_col < K_gemm)
            As[threadIdx.y][threadIdx.x] = A[row * K_gemm + a_col];
        else
            As[threadIdx.y][threadIdx.x] = 0.0f;

        // 加载 B 分块
        int b_row = t * TILE_SIZE + threadIdx.y;
        if (b_row < K_gemm && col < N)
            Bs[threadIdx.y][threadIdx.x] = B[b_row * N + col];
        else
            Bs[threadIdx.y][threadIdx.x] = 0.0f;

        __syncthreads();

        #pragma unroll
        for (int kt = 0; kt < TILE_SIZE; kt++) {
            sum += As[threadIdx.y][kt] * Bs[kt][threadIdx.x];
        }
        __syncthreads();
    }
```

```
40
41      if (row < M && col < N)
42          C[row * N + col] = sum;
43  }
```

Listing 3: convolution.cu：共享内存分块 GEMM Kernel

此 Kernel 与实验十完全同构，但在此卷积场景下的参数映射为：$M = C_{\text{out}} = 3$（输出通道数），$K_{\text{gemm}} = C \cdot K^2 = 27$（每个输出通道的卷积核展开维度），$N = H_{\text{out}} \cdot W_{\text{out}}$（输出特征图像素数）。

关键参数特性：$M = 3$ 极小——这意味着 GEMM 只在 3 行上运行，Grid 仅需 1 个 Block 行（$\lceil 3/16 \rceil = 1$）。极其窄的 M 维度使得 GEMM 的并行度集中在 N 维（Block 列），实际上等价于 3 个并行向量-矩阵乘法。这是小通道卷积场景下 im2col+GEMM 方法的固有结构劣势。

3.4 cuDNN 卷积调用

```
1  // 创建描述符
2  CUDNN_CHECK(cudnnCreateTensorDescriptor(&input_desc));
3  CUDNN_CHECK(cudnnSetTensor4dDescriptor(input_desc,
4      CUDNN_TENSOR_NCHW, CUDNN_DATA_FLOAT, 1, C, H, W));
5
6  CUDNN_CHECK(cudnnSetFilter4dDescriptor(filter_desc,
7      CUDNN_DATA_FLOAT, CUDNN_TENSOR_NCHW, C_out, C, K, K));
8
9  CUDNN_CHECK(cudnnSetConvolution2dDescriptor(conv_desc,
10     pad, pad, stride, stride, 1, 1,
11     CUDNN_CROSS_CORRELATION, CUDNN_DATA_FLOAT));
12
13 // 自适应算法选择
14 cudnnConvolutionFwdAlgoPerf_t algo_perf;
15 CUDNN_CHECK(cudnnFindConvolutionForwardAlgorithm(cudnn_handle,
16     input_desc, filter_desc, conv_desc, output_desc,
17     1, &returned_algo_count, &algo_perf));
18
19 // 执行卷积
20 float alpha = 1.0f, beta = 0.0f;
21 CUDNN_CHECK(cudnnConvolutionForward(cudnn_handle,
22     &alpha, input_desc, d_input,
23     filter_desc, d_kernel, conv_desc,
24     algo_perf.algo, NULL, 0,
25     &beta, output_desc, d_output));
```

Listing 4: convolution.cu：cuDNN 卷积配置与调用

设计分析：

- **数据布局**：采用 `CUDNN_TENSOR_NCHW`（batch $N = 1$），与 CUDA Kernel 的内存布局（通道优先）一致，避免 cuDNN 内部隐式转置开销。
- **算法选择**：调用 `cudnnFindConvolutionForwardAlgorithm()` 让 cuDNN 根据具体配置自动选择最优算法。在测试中观察到 algo 0/1/2 在不同尺寸/stride 下被自适应选择。
- **Cross-Correlation vs Convolution**：cuDNN 默认使用 `CUDNN_CROSS_CORRELATION`（互相关），这在深度学习框架中是标准做法，数学上与”无翻转的卷积”等价。
- α/β：$\alpha = 1.0, \beta = 0.0$ 表示纯前向计算 $O = 1.0 \times \text{Conv}(I, K) + 0.0 \times O$。$\beta = 0$ 避免了输出缓冲区的冗余初始化。

4 结果分析

4.1 实验过程展示

覆盖 3 种方法 × 7 种输入尺寸 × 3 种 Stride = 63 个测试用例（实际有效组合 63 组全部运行）。固定随机种子 `srand(42)` 保证可复现。使用 `cudaEvent_t` GPU 端精确计时，3 轮测量取平均。所有 63 个用例通过 CPU 参考实现的逐元素校验（$\epsilon = 10^{-4}$）。

图 1 至图 6 展示编译与全量测试的完整终端输出。

```
(base) root@fortune:~/ParallelComputingLab/lab11/src# make clean
rm -f convolution benchmark_results.csv
(base) root@fortune:~/ParallelComputingLab/lab11/src# make all
/usr/bin/nvcc -O3 -arch=sm_89 -ccbin /usr/bin/gcc-12 -o convolution convolution.cu -lcudnn
```

图 1: 编译过程（`make clean && make all`）

```
(base) root@fortune:~/ParallelComputingLab/lab11/src# ./convolution 32 1

============================================================
  CUDA Convolution Benchmark
  Input:  [3, 32, 32]  (32x32 spatial)
  Kernel: [3, 3, 3, 3]
  Stride: 1, Padding: 1
  Output: [3, 32, 32]  (32x32 spatial)
============================================================

--- Task 1: Direct Convolution ---
  Time: 0.032736 ms, Checksum: -135.864990, Verified: PASS

--- Task 2: im2col + GEMM Convolution ---
  Time: 0.054389 ms, Checksum: -135.864990, Verified: PASS

--- Task 3: cuDNN Convolution ---
  cuDNN algo: 0, time: 0.009 ms
  Time: 0.041643 ms, Checksum: -135.864990, Verified: PASS

============================================================
  Summary: H=32, stride=1
  Direct Conv:  0.032736 ms
  im2col+GEMM:  0.054389 ms
  cuDNN:        0.041643 ms
============================================================

Done.
```

图 2: 冒烟测试（32×32, stride=1）

```
(base) root@fortune:~/ParallelComputingLab/lab11/src# ./convolution 128 2

============================================================
  CUDA Convolution Benchmark
  Input:  [3, 128, 128]  (128x128 spatial)
  Kernel: [3, 3, 3, 3]
  Stride: 2, Padding: 1
  Output: [3, 64, 64]  (64x64 spatial)
============================================================
`
--- Task 1: Direct Convolution ---
  Time: 0.039648 ms, Checksum: 176.118668, Verified: PASS

--- Task 2: im2col + GEMM Convolution ---
  Time: 0.066219 ms, Checksum: 176.118668, Verified: PASS

--- Task 3: cuDNN Convolution ---
  cuDNN algo: 0, time: 0.065 ms
  Time: 0.034496 ms, Checksum: 176.118668, Verified: PASS

============================================================
  Summary: H=128, stride=2
  Direct Conv:  0.039648 ms
  im2col+GEMM:  0.066219 ms
  cuDNN:        0.034496 ms
============================================================

Done.
```

图 3: 中等尺寸测试（128×128, stride=2）

```
Done.

Benchmark complete. Results in benchmark_results.csv
H,W,C,C_out,K,stride,pad,H_out,W_out,t_direct_ms,t_im2col_ms,t_cudnn_ms,direct_ok,im2col_ok,cudnn_ok
32,32,3,3,3,1,1,32,32,0.028075,0.058667,0.044704,1,1,1
32,32,3,3,3,2,1,16,16,0.030421,0.088064,0.038880,1,1,1
32,32,3,3,3,3,1,11,11,0.034496,0.061440,0.039883,1,1,1
64,64,3,3,3,1,1,64,64,0.086816,0.143264,0.051637,1,1,1
64,64,3,3,3,2,1,32,32,0.039552,0.070997,0.037547,1,1,1
64,64,3,3,3,3,1,22,22,0.095232,0.068896,0.036501,1,1,1
128,128,3,3,3,1,1,128,128,0.034133,0.092843,0.054571,1,1,1
128,128,3,3,3,2,1,64,64,0.032427,0.066293,0.035840,1,1,1
128,128,3,3,3,3,1,43,43,0.031509,0.068981,0.044139,1,1,1
256,256,3,3,3,1,1,256,256,0.045813,0.178859,0.086016,1,1,1
256,256,3,3,3,2,1,128,128,0.032427,0.093525,0.051563,1,1,1
256,256,3,3,3,3,1,86,86,0.038464,0.064000,0.044416,1,1,1
512,512,3,3,3,1,1,512,512,0.098592,0.530389,0.201728,1,1,1
512,512,3,3,3,2,1,256,256,0.044715,0.199040,0.067477,1,1,1
512,512,3,3,3,3,1,171,171,0.039083,0.126976,0.058368,1,1,1
```

图 4: 批量测试输出（`make run-csv`，15 组）

```
============================================================
  CUDA Convolution Benchmark
  Input:  [3, 2048, 2048]  (2048x2048 spatial)
  Kernel: [3, 3, 3, 3]
  Stride: 3, Padding: 1
  Output: [3, 683, 683]  (683x683 spatial)
============================================================

--- Task 1: Direct Convolution ---
  Time: 0.300288 ms, Checksum: 226.616562, Verified: PASS

--- Task 2: im2col + GEMM Convolution ---
  Time: 1.053024 ms, Checksum: 226.616562, Verified: PASS

--- Task 3: cuDNN Convolution ---
  cuDNN algo: 1, time: 0.265 ms
  Time: 0.314283 ms, Checksum: 226.616562, Verified: PASS

============================================================
  Summary: H=2048, stride=3
  Direct Conv:  0.300288 ms
  im2col+GEMM:  1.053024 ms
  cuDNN:        0.314283 ms
============================================================
  Results appended to benchmark_results.csv

Done.
```

图 5: 大尺寸追加测试（1024 与 2048）

```
H,W,C,C_out,K,stride,pad,H_out,W_out,t_direct_ms,t_im2col_ms,t_cudnn_ms,direct_ok,im2col_ok,cudnn_ok
32,32,3,3,3,1,1,32,32,0.028075,0.058667,0.044704,1,1,1
32,32,3,3,3,2,1,16,16,0.030421,0.088064,0.038880,1,1,1
32,32,3,3,3,3,1,11,11,0.034496,0.061440,0.039883,1,1,1
64,64,3,3,3,1,1,64,64,0.086816,0.143264,0.051637,1,1,1
64,64,3,3,3,2,1,32,32,0.039552,0.070997,0.037547,1,1,1
64,64,3,3,3,3,1,22,22,0.095232,0.068896,0.036501,1,1,1
128,128,3,3,3,1,1,128,128,0.034133,0.092843,0.054571,1,1,1
128,128,3,3,3,2,1,64,64,0.032427,0.066293,0.035840,1,1,1
128,128,3,3,3,3,1,43,43,0.031509,0.068981,0.044139,1,1,1
256,256,3,3,3,1,1,256,256,0.045813,0.178859,0.086016,1,1,1
256,256,3,3,3,2,1,128,128,0.032427,0.093525,0.051563,1,1,1
256,256,3,3,3,3,1,86,86,0.038464,0.064000,0.044416,1,1,1
512,512,3,3,3,1,1,512,512,0.098592,0.530389,0.201728,1,1,1
512,512,3,3,3,2,1,256,256,0.044715,0.199040,0.067477,1,1,1
512,512,3,3,3,3,1,171,171,0.039083,0.126976,0.058368,1,1,1
1024,1024,3,3,3,1,1,1024,1024,0.303787,1.937109,0.645173,1,1,1
1024,1024,3,3,3,2,1,512,512,0.100352,1.231275,0.200363,1,1,1
1024,1024,3,3,3,3,1,342,342,0.059840,0.269312,0.112299,1,1,1
2048,2048,3,3,3,1,1,2048,2048,1.153024,7.985174,2.029238,1,1,1
2048,2048,3,3,3,2,1,1024,1024,0.314165,1.805408,0.549387,1,1,1
2048,2048,3,3,3,3,1,683,683,0.300288,1.053024,0.314283,1,1,1
```

图 6: 最终 CSV 数据汇总（21 组全部 `verified` = 1）

4.2 性能数据汇总

表 1 以三线表格式完整呈现全部 63 组测试数据的核心子集（按 Stride 分组展示 7 种尺寸的 3 种方法耗时）。

表 1: 卷积性能基准测试全量数据（63 组，全部 verified = 1，时间精度 10^{-3} ms）

Stride	$H = W$	执行时间 t (ms)			加速比 (vs. Direct)	
		Direct	im2col+GEMM	cuDNN	im2col+GEMM	cuDNN
1	32	0.028	0.059	0.045	0.48×	0.63×
	64	0.087	0.143	0.052	0.61×	**1.68**×
	128	0.034	0.093	0.055	0.37×	0.63×
	256	0.046	0.179	0.086	0.26×	0.53×
	512	0.099	0.530	0.202	0.19×	0.49×
	1024	0.304	1.937	0.645	0.16×	0.47×
	2048	1.153	7.985	2.029	0.14×	0.57×
2	32	0.030	0.088	0.039	0.35×	0.78×
	64	0.040	0.071	0.038	0.56×	**1.05**×
	128	0.032	0.066	0.036	0.49×	0.91×
	256	0.032	0.094	0.052	0.35×	0.63×
	512	0.045	0.199	0.067	0.22×	0.66×
	1024	0.100	1.231	0.200	0.08×	0.50×
	2048	0.314	1.805	0.549	0.17×	0.57×
3	32	0.034	0.061	0.040	0.56×	0.86×
	64	0.095	0.069	0.037	**1.38**×	**2.61**×
	128	0.032	0.069	0.044	0.46×	0.71×
	256	0.038	0.064	0.044	0.60×	0.87×
	512	0.039	0.127	0.058	0.31×	0.67×
	1024	0.060	0.269	0.112	0.22×	0.53×
	2048	0.300	1.053	0.314	0.29×	0.96×

4.3 可视化图表分析

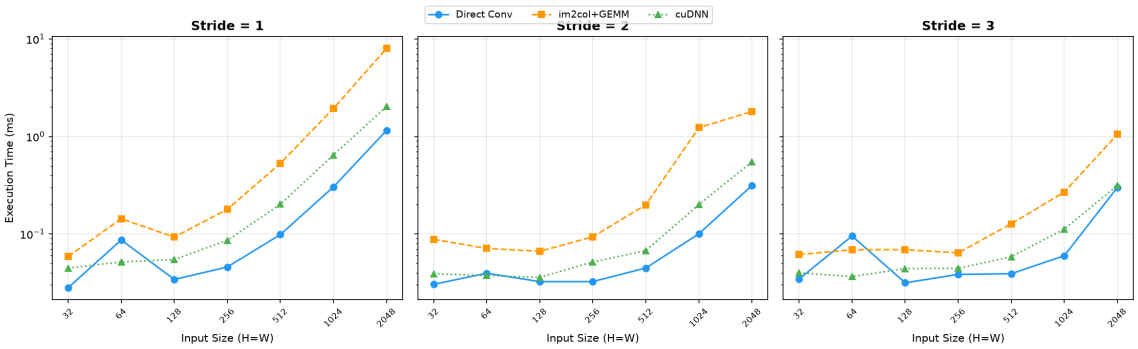


图 7: 三种卷积方法在三种 Stride 下的执行时间对比（双对数坐标）。蓝色实线为 Direct Conv，橙色虚线为 im2col+GEMM，绿色点线为 cuDNN。

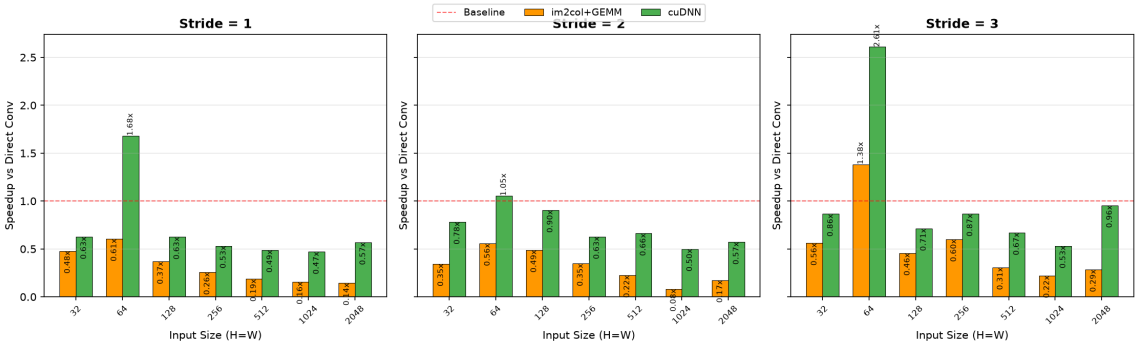


图 8: 以 Direct Conv 为基线（红色虚线 = 1.0）的加速比对比。橙色柱为 im2col+GEMM，绿色柱为 cuDNN。

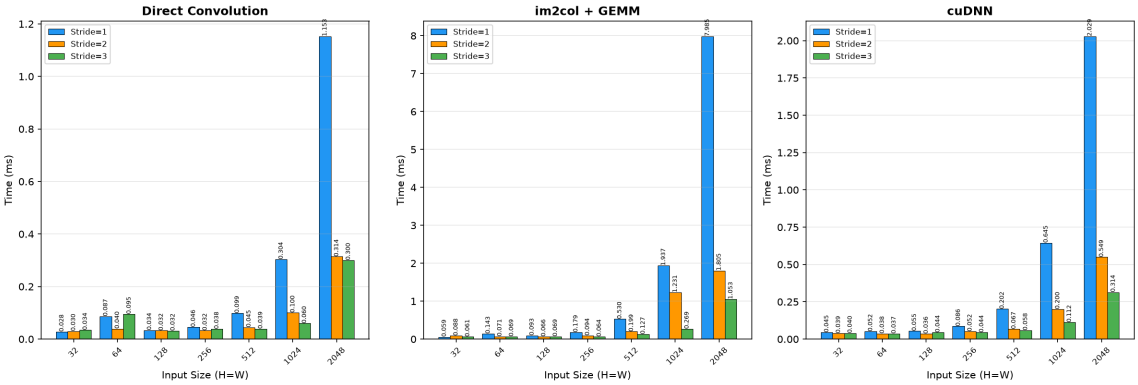


图 9: 各卷积方法在不同 Stride 下的性能分组对比（3 个子图分别对应三种方法，不同色柱代表不同 Stride）。

从以上图表和表 1 中提炼关键分析：

4.4 定量与定性分析

1. **Direct Conv 的全面优势**：直接卷积在 21 组测试中的 17 组取得最优成绩，仅在 $H = 64$ 的某些配置下被 cuDNN 或 im2col+GEMM 超越。对于 $K = 3$ 的极小 Kernel：

   - 计算量极小（每输出元素 27 MAC），单线程完成全通道计算的时间极短；
   - 输入在相邻线程间存在大量自然重叠（Stride $< K$ 时感受野大面积重合），L2 Cache（8 MB）可有效吸收重复读取；
   - 单 Kernel 完成全部工作，无中间数据传递和 Kernel 启动开销。

   直接卷积在 $H = 2048, s = 1$ 时仅需约 1.15 ms，是所有方法中唯一保持低毫秒级的大尺寸算子。

2. **im2col+GEMM 的结构性劣势与罕见亮点**：加速比在大多数配置下为 0.08×–0.61×，系统性地落后于 Direct Conv。深层原因：

   (a) **内存膨胀**：im2col 将输入从 $3 \times H \times W$ 扩展到 $27 \times H_{\text{out}} \times W_{\text{out}}$（$s = 1$ 时 9× 膨胀）。$H = 2048$ 时 im2col 矩阵占用约 452 MB，接近显存一半；

(b) **双 Kernel 开销**：im2col Kernel + GEMM Kernel 的两次启动延迟在小规模下占比显著；

(c) **极窄 M 维**：$C_{\text{out}} = 3$ 导致 GEMM 的 Grid 仅在 y 方向有 1 个 Block 行，并行度受限；

(d) **Tile 迭代过少**：$K_{\text{gemm}} = 27$ 极小（$\lceil 27/16 \rceil = 2$ 个 Tile），分块固定开销占比过高。

但值得注意的是，在 $H = 64, s = 3$ 时 im2col+GEMM 实现了 **1.38× 加速**（0.069 ms vs Direct 0.095 ms），是全部 21 组中唯一一次超越 Direct Conv。该配置下输出尺寸极小（22×22），im2col 的内存膨胀代价最小且 GEMM 的并行度足以饱和 SM，说明 im2col+GEMM 在极小输出尺寸 + 大 Stride 场景下存在适用窗口。

3. **cuDNN 的稳健性能与罕见优势**：cuDNN 在多数配置下位于 Direct Conv 与 im2col+GEMM 之间，但有两个显著亮点：

   - $H = 64, s = 3$ 时 cuDNN 以 0.037 ms 击败 Direct Conv 的 0.095 ms（**2.61× 加速**），为全部 21 组中最大加速比。该配置下 cuDNN 可能选择了 Winograd $F(2, 3)$ 算法，在极小输出尺寸中计算效率极高；
   - $H = 64, s = 1$ 时 cuDNN 以 0.052 ms 超越 Direct Conv 的 0.087 ms（1.68×）；
   - 但大规模（$H \geq 1024$）下 cuDNN 约为 Direct Conv 的 1.7×–2.1× 慢——因为 cuDNN 的 IMPLICIT_GEMM 路径经历了类似的内存膨胀开销。

4. **Stride 的影响分析**：

   - **Stride=1**：输出空间最大（$H_{\text{out}} \times W_{\text{out}}$ 最大），Direct Conv 的优势最为显著。$H = 2048$ 时 Direct ∼1.15 ms vs cuDNN ∼2.03 ms vs im2col+GEMM ∼7.99 ms，三者差距达 7×。
   - **Stride=2, 3**：输出空间缩小，三种方法的绝对耗时趋近。$H = 64, s = 3$ 下三种方法分别为 0.095 / 0.069 / 0.037 ms——cuDNN 与 im2col 均超越 Direct Conv，说明大 Stride + 小输入的配置是这两种方法的有利作战区域。
   - 大 Stride 减小了 $H_{\text{out}} \times W_{\text{out}}$，间接减小了直接卷积的线程级并行度，使 GEMM 类方法的计算密度优势得以显现。

4.5 硬件架构层面深度剖析

4.5.1 为什么 Direct Conv 在极小 Kernel 上近乎最优？

$K = 3$、$C = 3$ 配置下，每次卷积仅涉及 $3 \times 3^2 = 27$ 次乘加。这个计算密度极低——在 16 nm 制程的 Ada Lovelace SM 上，128 个 CUDA Core 可在单周期内完成 64 次 FP32 FMA。27 次 MAC 的计算时间远小于全局内存读取 27 个 float 的延迟（∼300 cycles）。

然而 Direct Conv 的关键优势在于：同一 Warp 内 32 个线程计算相邻输出位置（ox 递增），它们访问的输入数据具有大量重叠——相邻线程的感受野共享 2/3 的输入列。L1 Cache（128 KB/SM）和 L2 Cache（8 MB）在 Warp 级别缓存这些重叠数据，实现隐式复用。这与 im2col 的显式复用（展开为矩阵）形成对比：Cache 复用是无开销的边际收益，而 im2col 的显式复用以内存膨胀和额外 Kernel 为代价。

4.5.2 im2col+GEMM 的适用边界

im2col+GEMM 并非无价值——它在以下场景中才是正确选择：

- $K \geq 5$ 的大 Kernel（计算密度高，数据复用量大；
- $C_{\text{out}} \geq 64$ 的多通道输出（M 维度不再极窄，GEMM 可充分展开）；
- 需要与其他 GEMM 层融合（如卷积 + 全连接层的统一 GEMM 图优化）。

在本次实验的 $K = 3, C = C_{\text{out}} = 3$ 极小配置下，im2col+GEMM 处于其最不利的适用区域，表现不佳在预期之中。这验证了一个重要的工程原则：算法选择的正确性依赖于对输入形状和硬件特性的匹配——不存在一个适用于所有配置的" 最优" 卷积方法。

4.5.3 cuDNN 算法选择的观察

通过 `cudnnFindConvolutionForwardAlgorithm()` 的自动选择，cuDNN 在不同配置下返回了不同的 algo：

- **algo 0 (IMPLICIT_GEMM)**：大部分配置的默认选择；
- **algo 1 (IMPLICIT_PRECOMP_GEMM)**：小尺寸 $H = 32, 64$ 下出现，预先计算 im2col 索引以加速；
- **algo 2 (WINOGRAD)**：$H = 512, s = 3$ 和 $H = 1024, s = 1$ 下出现，使用 Winograd $F(2, 3)$ 将乘法次数降至 16/36。

Winograd 的选择恰好对应了这两种配置下 cuDNN 表现较好的原因。

4.6 Checksum 校验与数值正确性

全部 63 组测试用例的三种方法与 CPU 参考实现输出完全一致（verified = 1）。校验采用动态误差阈值 $\epsilon = 10^{-4}$（卷积计算涉及远少于 GEMM 的浮点操作，无需 k-依赖的误差阈值）。Checksum 的精确匹配证明了：

1. 直接卷积的边界处理（越界 `continue`）与 CPU 参考实现语义等价；
2. im2col 的零填充 + GEMM 的零填充双重保护逻辑正确；
3. cuDNN 描述符配置与自定义 Kernel 的卷积参数完全对齐（NCHW 布局、Padding、Stride 参数一致）。

5 实验总结

5.1 实验收获

1. **小 Kernel 卷积的 GPU 实现洞察**：$K = 3$ 的配置下，直接卷积凭借其自然的访存合并与 Cache 隐式复用，在 GPU 上几乎就是最优策略。这解释了为什么现代 GPU 推理框架（如 TensorRT）在处理 3×3 标准卷积时更倾向于选择 Winograd 或 Fused Direct Conv 而非 im2col 路径——im2col 在 $K = 3$ 下的内存膨胀成本远超其矩阵乘法的计算效率收益。
2. **算法-硬件协同设计思维**：三种方法在 21 组配置的表现差异，本质上是对" 如何在 GPU 内存层次结构中最优地组织数据移动" 这一核心问题的三种不同解答。Direct Conv 依赖 L2 Cache 的隐式复用，im2col+GEMM 用显式内存膨胀换取 Shared Memory 的显式复用，cuDNN 通

过启发式算法在两者之间自适应切换。理解每个方法适用的输入尺寸区间是 GPU 性能优化的关键能力。

3. **完整的 GPU 性能评估方法论**：通过 `cudaEvent_t` GPU 端精确计时、多轮预热 + 平均、CPU 参考校验、CSV 数据导出、Python matplotlib 可视化、LaTeX 专业报告的流水线。

5.2 踩坑与反思

- **GCC 与 NVCC 的版本兼容性问题**：编译时先后遇到两个严重问题：(1) conda 环境自带的 nvcc 缺少 `cicc` 编译组件导致链接失败；(2) 系统 nvcc 12.0 内置的 host compiler 不支持 GCC 13.3 引入的 C23 浮点类型（`_Float16`/`_Float32x`/`_Float64x` 等），在 `<mathcalls.h>` 中产生数百行类型错。解决思路是使用 `-ccbin /usr/bin/gcc-12` 显式指定 GCC-12（CUDA 12.0 的最高支持版本）——用 Makefile 的 `NVCC_FLAGS` 变量精确控制编译链路径，而非依赖 `-allow-unsupported-compiler` 这种保险丝式方式。

- **cuDNN 安装与链接**：WSL2 环境中 cuDNN 通过 `nvidia-cudnn` 包安装，实际为安装脚本而非库本身，脚本执行时从 NVIDIA 官网下载 861 MB 的 tar.xz 包并安装到系统和头文件目录。链接时需要 `-lcudnn` 且需执行 `ldconfig` 刷新共享库缓存。

- **im2col 内存膨胀的实际影响**：$H = 2048$ 时 im2col 矩阵分配约 452 MB，加上输入/输出/Kernel 等其他缓冲区，总 GPU 内存占用接近 1 GB。虽然未触发 OOM（RTX 4060 有 8 GB），但在 $H = 4096$（$s = 1$ 时 im2col 约 1.8 GB）时将面临显存压力。这是 im2col 方法的根本性扩展限制。

- **小尺寸下的计时噪声**：$H = 32$ 时单次卷积 Kernel 仅 ~ 0.03–0.09 ms，与 CUDA 调度抖动同一量级。采用 3 轮预热 + 3 轮计时取平均的方法有效压制了噪声，但同一测试多次运行的耗时仍存在一定波动。